Proposal, version 4

A simple score for a run (`sec:pf:measures`)

For review before anything is applied

5 October 2026

Version 3 replaced the weighted criterion $\mathcal{J}$ with four questions. Version 4 simplifies further, following your last answers. Traffic concentration is removed, and congestion is measured by crowding alone. Robustness is no longer a separate term. The slow loop gets one count of its own. Versions 2 and 3 are kept as `scoring_proposal_v2.pdf` and `scoring_proposal_v3.pdf`. Nothing in the thesis has been changed, and the figures are drafts.

Contents

1 Four principles

Every measure in the score follows from these four principles. A measure that no principle asks for is left out.

1. **The score judges the whole warehouse, not each layer.** Each layer already has its own objective, and those objectives are tools for producing good behaviour. The RL reward serves the fast loop, and the storage objective (`eq:storageobjective`) serves the slow loop. The score judges what both layers achieve together.

2. **The lifelong MAPD measure decides.** The result is mean service time, as in **?**, and it is valid only if the fleet keeps up with the arriving tasks.

3. **Explanations add up to the result.** Service time splits exactly into three parts: waiting for an agent, travelling, and being blocked. A change in the result can always be traced to the part that moved. The only other explanation is one congestion reading.

4. **Every measure is a count or an average, and every source is named.** A measure taken from a paper is used as that paper defines it and is not claimed as a contribution. Where the thesis adds something, the card says what, and only that part is the thesis's own responsibility.

2 The score at a glance

	Measure	Role	Source
1	keeping up: throughput against arrival rate, and backlog	check	**?**
2	mean service time $\bar{\zeta}_T$	**result**	**?**
3	its three parts: waiting, travel, blocked	explanation	own split
4	crowding $\bar{\delta}_T$	congestion	CRAMP-style density
5	units relocated per storage update $\bar{\nu}_T$	slow-loop cost	own
6	runtime per timestep κ_T	reported	**?**

Every result is reported as a mean with a 95% confidence interval over evaluation seeds. Section **??** explains why this replaces a separate robustness measure.

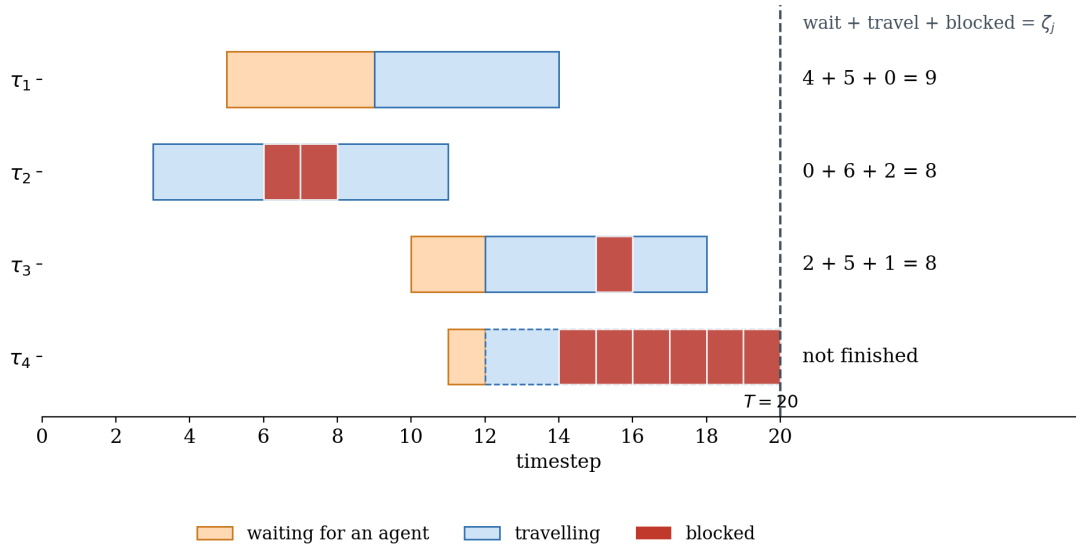


Figure 1: Four tasks over a horizon $T = 20$, with each service time split into its three parts. τ_4 is stuck from $t = 14$ and never finishes.

Figure **??** is the worked example for measures 1 to 3. τ_1 is the task of `fig:taskroute`. Agent a_2 is assigned at $t = 9$ and moves one edge per timestep to D1, which it reaches at $t = 14$, so its parts are $4 + 5 + 0$.

3 The measures

1. Keeping up: throughput against arrival rate, and backlog

Question. Is the fleet finishing tasks as fast as they arrive?

What it counts. Throughput is the number of tasks finished per timestep. The fleet keeps up when throughput stays close to the arrival rate. The backlog, the number of tasks released but not yet finished, is logged at every timestep and plotted, so a run that falls behind can be seen.

Formula.

$$\Lambda_T = \frac{|\mathcal{C}_T|}{T}, \qquad \text{the run keeps up if } \Lambda_T \geq f_{\text{up}}\,\lambda_{\text{task}}, \qquad B_t = |\mathcal{Q}_t| + |\mathcal{B}_t|.$$

Worked number. Four tasks are released and three finish within $T = 20$. Throughput is $3/20 = 0.15$ per timestep against $4/20 = 0.20$ released, a ratio of 0.75. The backlog ends at $B_{20} = 1$, the stuck task τ_4.

Source. **?** count a lifelong MAPD instance as solved only if the service time of all tasks stays bounded, which means the fleet keeps up. Throughput as completed tasks per unit time is the pick rate of **?**. *Own part:* the ratio test and its threshold f_{up}.

Role. A check, applied first. A run that does not keep up is reported as such and is not compared on service time, because service time only counts finished tasks. This is also how a stuck task is caught.

2. Mean service time $\bar{\zeta}_T$, the result

Question. On average, how long does a task take from its release to its delivery?

What it counts. For each finished task, the number of timesteps from its release to its delivery, averaged over the finished tasks.

Formula.

$$\bar{\zeta}_T = \frac{1}{|\mathcal{C}_T|} \sum_{\tau_j \in \mathcal{C}_T} \zeta_j, \qquad \zeta_j = t_j^{\text{finish}} - t_j^{\text{released}} \quad \texttt{(eq:servicetime)}.$$

Worked number. τ_1, τ_2 and τ_3 take 9, 8 and 8 timesteps, so $\bar{\zeta}_T = 25/3 \approx 8.33$.

Source. **?** define service time as "the average number of timesteps … needed to finish executing each task after it was added to the task set", and use it to evaluate a MAPD algorithm. The thesis uses it unchanged. *Own part:* none.

Role. The result. Adaptive storage beats fixed storage under a controller when both runs keep up and the confidence intervals of $\bar{\zeta}_T$ do not overlap.

3. The three parts of service time

Question. Where did the time go?

What it counts. Every timestep of a task's life belongs to exactly one of three parts. Before assignment the task *waits for an agent*. After assignment its agent either moves, which is *travel*, or stands still, which is *blocked*. Standing still covers both a chosen wait and a move that conflict resolution turned into a wait. The three parts therefore always add up to the service time.

Formula. For a finished task τ_j,

$$\zeta_j = \underbrace{t_j^{\text{assigned}} - t_j^{\text{released}}}_{\text{waiting for an agent}} + \underbrace{\zeta_j^{\text{travel}}}_{\text{travel}} + \underbrace{\sum_{t=t_j^{\text{assigned}}}^{t_j^{\text{finish}}-1} \omega_j(t)}_{\text{blocked}},$$

with the travel part the number of moves. Each part is averaged over the finished tasks like $\bar{\zeta}_T$, so the three means add up to $\bar{\zeta}_T$. The blocked mean is W_T of `eq:waiting`, unchanged.

Worked number. The three tasks give means of $(4+0+2)/3 = 2$ waiting, $(5+6+5)/3 \approx 5.33$ travel and $(0+2+1)/3 = 1$ blocked, which add up to 8.33.

Source. The service time is that of **?**. *Own part:* the split into three parts. It needs no new data, because the Method already logs the assignment time and the override flag (`sec:impl:logging`).

Role. Explains the result. Each part points to one cause.

- Waiting for an agent grows when the fleet is too small or poorly assigned.
- Travel shrinks when storage puts popular items close to where they are needed. This is where adaptive storage should show, so it takes over the role movement cost had in RQ1.
- Blocked time grows with congestion.

4. Crowding $\bar{\delta}_T$, the congestion measure

Question. How crowded was the space around the agents?

What it counts. Each agent's congestion feature $\delta_i(t)$, the share of its field of view occupied by other agents, averaged over every agent and every timestep.

Formula.

$$\bar{\delta}_T = \frac{1}{mT} \sum_{t=0}^{T-1} \sum_{i=1}^{m} \delta_i(t) \qquad (\delta_i(t) \text{ from } \texttt{eq:congestion}).$$

Worked number. In the centralised panel of `fig:congestion` (Figure **??**), the three agents read 0, 0.25 and 0.25. Averaged, that timestep contributes $0.5/3 \approx 0.17$. Averaging such timesteps over the horizon gives $\bar{\delta}_T$.

Source. The per-agent density follows CRAMP (**?**), which counts the agents in an agent's field of view and divides by the free space there. Three differences are stated, so the thesis calls it "CRAMP-style" and not CRAMP's measure.

- CRAMP uses the density only in its reward, with a threshold above which an area counts as crowded. It does not report the density as a result.
- CRAMP measures it over a region smaller than its field of view. The thesis uses the field of view d_{obs} itself.
- The thesis averages it into one reported number, with no threshold.

Own part: the averaging into an outcome.

Role. The congestion measure, and the answer to RQ3. Crowding and blocked time are read together. Crowding with no blocking costs nothing, as with a convoy that never stops. Crowding together with blocking is congestion that costs time.

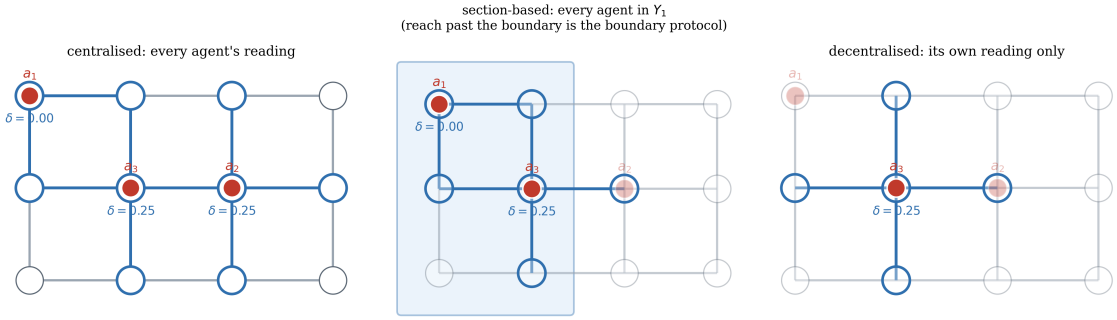


Figure 2: The existing `fig:congestion`, used for the worked number of the crowding measure.

5. Units relocated per storage update $\bar{\nu}_T$, the slow-loop cost

Question. How much stock does adaptive storage move?

What it counts. At every storage update, the number of item units moved to a new location. Averaged over the updates in the horizon. Fixed storage always scores 0.

Formula. With U storage updates in the horizon and r_u the units relocated at update u,

$$\bar{\nu}_T = \frac{1}{U} \sum_{u=1}^{U} r_u \;\le\; \nu,$$

where ν is the cap on relocations per update from the Method.

Worked number. With $\Delta = 50$ and $T = 1000$ there are $U = 20$ updates. If they move 120 units in total, $\bar{\nu}_T = 6$ units per update. These values only serve the example.

Source. No paper in the folder reports relocation. *Own part:* the whole measure.

Role. Reported next to the result. Service time already shows what adaptive storage gains. This count shows what it costs, which service time cannot see, because the model does not simulate moving the stock.

6. Runtime per timestep κ_T

Question. How long does the controller take to choose the next joint action?

What it counts. The wall-clock time of assignment and routing together at each timestep, averaged over the horizon. How it is measured is fixed in `sec:impl:cost`.

Formula.

$$\kappa_T = \frac{1}{T}\sum_{t=0}^{T-1} \kappa(t) \qquad (\texttt{eq:runtime},\ \text{unchanged}).$$

Worked number. If deciding takes 2 ms at every timestep, $\kappa_T = 2\,\text{ms}$.

Source. **?** report runtime per timestep and ask for it to stay below one second for real-time lifelong operation. *Own part:* none.

Role. Reported. It is measured in seconds, so it is never combined with service time.

4 Robustness, as other papers handle it

None of the papers in the folder measures robustness as a quantity of its own. They all report averages over many runs and vary the load.

Paper	Runs per setting	Load varied
?	one sequence of 500 tasks	6 task rates, 5 fleet sizes
CRAMP	average over 100 environments	4 fleet sizes, 4 obstacle densities
?	95% bootstrap confidence interval	two simulators, several layouts
?	5 seeds, mean and standard error	fleet size and map size

The proposal therefore removes "robustness" from the main RQ and adds no measure for it. Every result is reported as a mean with a 95% confidence interval over evaluation seeds, as **?** do. Varying the load is already RQ4. A reader who wants to know whether a gain is reliable reads it from the interval and from the load sweep.

5 The decision problem and the research question

The decision problem becomes one sentence. Choose the storage rule and the controller with the shortest expected service time among those that keep up. Formally,

$$(F^\star, \pi^\star) \in \arg\min_{F\in\mathcal{F},\,\pi\in\Pi} \mathbb{E}\big[\bar{\zeta}_T(F,\pi)\big] \quad \text{subject to keeping up and the constraints of the model.}$$

The expectation is over the order stream and the random seeds. The study estimates it by the mean over evaluation seeds.

The thesis does not solve this problem. It compares fixed and adaptive storage under each controller, so `eq:rqformal` reads

$$\mathbb{E}\big[\bar{\zeta}_T(F,\pi)\big] < \mathbb{E}\big[\bar{\zeta}_T(F_{\text{fix}},\pi)\big] \quad \text{with non-overlapping confidence intervals, both runs keeping up.}$$

6 Proposed wording of the research questions

The research questions name measures that version 4 removes, so they must change with it. The wording below is a proposal. The research questions are yours to decide.

Now. How does closed-loop coupling between storage assignment and MAPD control affect efficiency, congestion, traffic concentration, and robustness in a graph-based warehouse?

Proposed. How does closed-loop coupling between storage assignment and MAPD control affect service time and congestion in a graph-based warehouse?

Now. RQ1. Does closed-loop storage adaptation improve service time, throughput, or movement cost relative to fixed storage under otherwise matched conditions?

Proposed. RQ1. Does closed-loop storage adaptation shorten service time relative to fixed storage under otherwise matched conditions, and which part of service time changes?

Now. RQ2. How does the effect of coupling differ among centralised, section-based, and decentralised controllers that share one learning method and differ only in the information they condition on?

Proposed. Unchanged.

Now. RQ3. Do congestion-sensitive storage and routing policies reduce waiting and traffic concentration, and what efficiency trade-offs do they introduce?

Proposed. RQ3. Do congestion-sensitive storage and routing policies reduce crowding and blocked time, and does the travel they add cost more service time than they save?

Now. RQ4. How do these effects change as the number of agents and the task arrival rate increase?

Proposed. RQ4. How do these effects change as the number of agents and the task arrival rate increase, and at what load does the fleet stop keeping up?

Now. RQ5 (conditional), on local communication.

Proposed. Unchanged.

Now. Contribution (iii). A controlled empirical analysis of when feedback improves performance and when it amplifies congestion or concentrates traffic on a small part of the warehouse.

Proposed. Contribution (iii). A controlled empirical analysis of when feedback shortens service time and when it increases congestion.

Now. Dependent variables. Mean service time, throughput, movement cost per completed task, waiting caused by contention, traffic concentration, backlog, and controller runtime.

Proposed. Dependent variables. Whether the fleet keeps up (throughput against arrival rate, and backlog), mean service time and its three parts (waiting for an agent, travel, blocked), crowding, units relocated per storage update, and controller runtime.

7 What is removed and what it touches

Removed from the model.

- $\mathcal{J}(F, \pi)$ (`eq:functional`) and its five weights $q_\bullet$.
- Traffic concentration: H_T, C_T, E_T^+, $p_T(e)$ (`eq:entropy`, `eq:concentration`), the `[G8]` convention, and `fig:concentration`. The risk of funnelling traffic onto one aisle stays in the storage rule, through the congestion exposure term K of `eq:storageobjective`, and its effect shows in crowding and blocked time.

- Movement cost $\bar{c}_T$ (`eq:movementcost`) as a reported measure. The travel part of service time takes its place, in the same unit as the result. The one-step cost $\hat{c}$ itself stays, since other parts of the model use it.
- Robustness as a term in the main RQ.

Added. The split of service time, crowding $\bar{\delta}_T$, the relocation count $\bar{\nu}_T$ and the threshold f_{up}. $\bar{\nu}_T$ uses the letter of the cap ν, which it never exceeds. r_u is local to its formula and needs no row in the notation table.

Places in the thesis that change.

- The scoring section, rewritten as the six measures in the card layout.
- `eq:decisionproblem` and `eq:rqformal`.
- The main RQ, RQ1, RQ3, RQ4, contribution (iii) and the dependent variables, as in Section **??**.
- The scope paragraph on deadlines (`[K2]`), which refers to $\mathcal{J}$ and `eq:functional`.
- The opening of the Method chapter, which names $\mathcal{J}$.
- The instrumentation section. H_T is removed, and relocations per update are logged.
- The notation table.
- A new tag `[A20]` and a `GAPS.md` entry with the reasons above.

Not touched. The RL reward, the storage rule F and its objective, and the congestion feature $\delta_i(t)$ itself.

8 Your decisions

Your decisions.

1. Adopt the six measures and the four principles?
2. Adopt the new RQ wording, or adjust it?
3. Should the keep-up threshold f_{up} stay TBD in the Method tables, or do you have a value?
4. Should the comparison rule be non-overlapping 95% confidence intervals, as proposed, or a paired test over seeds? The paired test is stronger, because the order stream is identical under every storage rule (`sec:impl:instances`), but harder to explain.

Once you agree, the changes are applied as listed in Section **??**. The figure scripts then move to `thesis-progress/scripts/images/`, with outputs copied to both `Images/` folders, and nothing is committed before you have read the result.

References

Aleksandar Krnjaic, Raul D. Steleac, Jonathan D. Thomas, Georgios Papoudakis, Lukas Schäfer, Andrew Wing Keung To, Kuan-Ho Lao, Murat Cubuktepe, Matthew Haley, Peter Börsting, and Stefano V. Albrecht. Scalable multi-agent reinforcement learning for warehouse logistics with robotic and human co-workers, 2022. URL https://arxiv.org/abs/2212.11498.

Tim Tsz-Kit Lau and Biswa Sengupta. The multi-agent pickup and delivery problem: Mapf, marl and its warehouse applications. *arXiv preprint arXiv:2203.07092*, 2022. URL https://arxiv.org/abs/2203.07092. Version 1, submitted 14 March 2022. Accessed: 2026-06-08.

Hang Ma, Jiaoyang Li, T. K. Satish Kumar, and Sven Koenig. Lifelong multi-agent path finding for online pickup and delivery tasks. *Transportation Science*, 54(2):473–489, 2020. doi: 10.1287/trsc.2018.0826. URL https://doi.org/10.1287/trsc.2018.0826. Originally published in AAMAS 2017 proceedings; journal version accessed 2026-06-08.

Phu Pham and Aniket Bera. Optimizing crowd-aware multi-agent path finding through local communication with graph neural networks, 2024. URL https://arxiv.org/abs/2309.10275.